



İstanbul Teknik Üniversitesi Robotik Arama Kurtarma Ekibi
Görev Raporu

Görev Konusu: ROS2, ROS2 Elemanları ve Çarpışma Önleyici Algoritmalar

Hazırlayan: Ömer Faruk Aydın
Teslim Tarihi: 17 Temmuz 2026

ROS2 Tanıma Araştırması

ROS2 Nedir?

Robot İşletim Sistemi 2 (ROS2 - Robot Operating System), geleneksel bir işletim sistemi (Windows, Linux vb.) olmaktan ziyade, işletim sisteminin üzerinde çalışan bir ara katman yazılımıdır (middleware). Robotik sistemler geliştirmek için tasarlanmış açık kaynaklı bir kütüphane, araç ve donanım soyutlama (hardware abstraction) bütünüdür. ROS1'in eksikliklerini gidermek, endüstriyel standartlara uymak ve gerçek zamanlı (real-time) sistemleri desteklemek amacıyla DDS (Data Distribution Service) altyapısı üzerine inşa edilmiştir (ROS Documentation, n.d.; Macenski et al., 2022).

Robotikte Hangi Amaçla Kullanılır?

ROS2, donanım ile yazılım arasındaki karmaşık iletişimi standartlaştırmak için kullanılır. Bir robot; motorlar, kameralar, LIDAR'lar ve çeşitli sensörlerden oluşur. ROS2, bu farklı donanımların birbirleriyle ve yazılım algoritmalarıyla (SLAM, görüntü işleme, otonom navigasyon) standart bir protokol üzerinden konuşmasını sağlar. Geliştiricilere "tekerleği yeniden icat etmeme" imkanı sunarak, doğrudan robotun görevine ve mantığına odaklanmalarına olanak tanır.

Avantajları ve Dezavantajları

Avantajları:

- **Modülerlik:** Sistem "node" (düğüm) adı verilen küçük, bağımsız parçalara bölünmüştür. Bir modül çöktüğünde tüm robot durmaz.
- **DDS Altyapısı:** Merkezi bir sunucuya ihtiyaç duymadan (Peer-to-Peer) haberleşme sağlar. QoS (Quality of Service) profilleri ile veri iletiminin güvenilirliği ayarlanabilir.
- **Çoklu Dil ve Platform Desteği:** C++ ve Python gibi dilleri destekler. Linux, Windows ve macOS üzerinde çalışabilir.

Dezavantajları:

- **Öğrenme Eğrisi:** Kavramsal karmaşıklığı (DDS ayarları, QoS profilleri, launch yapıları) nedeniyle yeni başlayanlar için başlangıçta zorlayıcıdır.
- **Performans Yükü (Overhead):** Ara katman iletişimi, özellikle mikrodenetleyicilerde veya çok yüksek frekanslı işlemlerde sistem üzerinde ekstra bir işlem ve bellek yükü yaratabilir.

ROS ve Açık Kaynak Paket Kullanım Seviyesi

Bir projede ROS kullanımı "görev odaklı" olmalıdır. Standart ve topluluk tarafından rüştünü ispatlamış algoritmalar (örneğin; haritalama için slam_toolbox, navigasyon için Nav2) doğrudan açık kaynak paketler olarak kullanılmalıdır. Ancak robota özgü karar mekanizmaları, çekirdek kontrol algoritmaları ve donanıma özel güvenlik kısıtlamaları projenin kendi ekibi tarafından geliştirilmelidir. Temel kural; altyapı için ROS2 kullanmak, rekabet avantajı ve özgünlük gerektiren yerlerde kendi algoritmalarınızı (Custom Nodes) entegre etmektir.

ROS2 Kullanımı Nasıl Azaltılabilir?

Sistemin hantallaşmasını önlemek ve ROS2 bağımlılığını azaltmak için "Agnostik Kod" yazım tarzı benimsenmelidir. Temel veri yapıları (data structures) ve çekirdek algoritmalar, ROS'tan tamamen bağımsız saf C veya C++ kütüphaneleri olarak geliştirilmelidir. ROS2, sadece bu kütüphanelere veri taşıyan bir "haberleşme arayüzü" (wrapper) olarak konumlandırılmalıdır. Ayrıca, çok düşük seviyeli ve gerçek zamanlı motor kontrol döngülerinde ROS2 yerine doğrudan mikrodenetleyici üzerinde çalışan gömülü C yazılımları veya Micro-ROS kullanılabilir. Zero-Copy (Ortak Bellek) teknikleri ile ağ üzerindeki veri yükü azaltılabilir.

ROS2 Elemanlarını Öğrenme Araştırması

ROS2 Workspace (Çalışma Alanı)

- **Tanım:** Projeye ait tüm kaynak kodların, ROS2 paketlerinin, derleme dosyalarının ve bağımlılıkların barındırıldığı ana klasördür (dizindir).
- **Nasıl Yardımcı Olur?:** Geliştirilen farklı yazılımların düzenli bir şekilde tutulmasını, derlenmesini ve birbirleriyle çakışmadan sistematik bir şekilde yönetilmesini sağlayarak kod organizasyonunu sağlar.

Nodes (Düğümmler)

- **Tanım:** Tek bir işlevi veya görevi (örneğin kameradan görüntü alma, motorlara güç verme, harita çıkarma) yerine getiren, kendi başına çalışabilen bağımsız yazılım birimleridir.
- **Nasıl Yardımcı Olur?:** Sistemi modüler hale getirir. Bir hata oluştuğunda tüm robot sisteminin çökmesini engeller; sadece ilgili düğüm kapanır ve geri kalan işlemler çalışmaya devam edebilir.

Topics (Konular)

- **Tanım:** Düğümmler arasında sürekli ve tek yönlü veri akışını sağlayan yayıncı (publisher) ve abone (subscriber) tabanlı iletişim kanallarıdır.
- **Nasıl Yardımcı Olur?:** Sensör verilerinin (örneğin saniyede onlarca kez güncellenen LIDAR taramaları) veya sürekli hız komutlarının sistemdeki ilgili diğer düğümlere kesintisiz, hızlı ve eşzamanlı olarak aktarılmasını sağlar.

Services (Servisler)

- **Tanım:** Sürekli veri akışı yerine, yalnızca ihtiyaç duyulduğunda çağrılan "istek-cevap" (request-response) tabanlı iki yönlü haberleşme modelidir.
- **Nasıl Yardımcı Olur?:** Sensör kalibrasyonu yapma, haritayı kaydetme veya robotu belirli bir moda geçirme gibi anlık tetiklenmesi gereken görevlerde ağı sürekli meşgul etmeden işlem yapılmasını sağlar.

Master-Slave Structure (ROS2 Farkı)

- **Tanım:** ROS1'de bulunan merkezi sunucu (roscore) yapısıdır. ROS2'de bu yapı kaldırılarak, düğümlerin merkezi bir yönetici olmadan DDS (Data Distribution Service) altyapısı ile birbirini otomatik keşfettiği merkeziyetsiz (decentralized) bir sisteme geçilmiştir.
- **Nasıl Yardımcı Olur?:** Tek nokta hata riskini (Single Point of Failure) ortadan kaldırır. Ana sunucu çökse bile düğümler birbirleriyle haberleşmeye devam eder, böylece robotun çalışmasındaki güvenilirlik ve hata toleransı maksimuma çıkar.

Parameter Server

- **Tanım:** Düğümlerin çalışması için gereken konfigürasyon ayarlarının (maksimum hız, sensör hassasiyeti, PID katsayıları vb.) saklandığı genel veri alanıdır.
- **Nasıl Yardımcı Olur?:** Kaynak kodun içine müdahale edip yeniden derleme yapmaya gerek kalmadan, robotun davranışlarını çalışma anında (runtime) veya başlatma sırasında kolayca değiştirebilmemize imkan tanır.

Action Lib

- **Tanım:** Uzun süren görevlerde (örneğin otonom bir şekilde 50 metre ilerideki hedefe gitme), işlemin durumunu bildiren (feedback) ve gerektiğinde iptal edilebilen bir iletişim yapısıdır.
- **Nasıl Yardımcı Olur?:** Robotun hedefe ulaşma yüzdesi veya anlık konumu gibi ilerleme durumlarını takip etmemizi ve ortamda beklenmedik bir engel çıktığında görevi iptal edip yeni komutlar verebilmemizi sağlar.

ROS2 .launch files

- **Tanım:** Robotu ayağa kaldırırken birden fazla düğümü, önceden belirlenmiş parametreler ve belirli bir düzenle aynı anda başlatmamızı sağlayan Python veya XML tabanlı betik dosyalarıdır.
- **Nasıl Yardımcı Olur?:** Geliştiriciyi onlarca farklı terminal açıp her bir düğümü tek tek çalıştırma zahmetinden kurtarır. Tek bir komutla tüm sistemi eksiksiz ve kullanıma hazır bir şekilde başlatmayı sağlar.

ROS2 .yaml files

- **Tanım:** Parametreleri ve yapılandırma ayarlarını insanlar tarafından kolayca okunabilen ve düzenlenebilen bir metin formatında saklayan dosyalardır.
- **Nasıl Yardımcı Olur?:** Robotun karmaşık ayar listelerini düzenli bir dosyada saklayarak, sistem kurulumunun ve başka robotlara kopyalanmasının çok daha kolay olmasını sağlar.

CMakeLists.txt

- **Tanım:** C++ ile yazılmış paketlerin nasıl derleneceğini, hangi kütüphanelere bağımlı olduğunu ve hangi dosyaların çalıştırılabilir (executable) olacağını tanımlayan yapılandırma dosyasıdır.
- **Nasıl Yardımcı Olur?:** Kodun donanımdan ve işletim sisteminin karmaşık derleme komutlarından bağımsız olarak, doğru kütüphanelerle otomatik ve eksiksiz bir biçimde derlenmesini garanti eder.

ROS2 MoveIt Package

- **Tanım:** Kinematik hesaplamalar, yörünge planlaması, 3 boyutlu ortam algısı ve çarpışma önleme algoritmalarını bir arada sunan gelişmiş bir hareket planlama kütüphanesidir.
- **Nasıl Yardımcı Olur?:** Karmaşık fiziksel matematik hesaplamalarını geliştiricinin yerine yaparak, robotun bir noktadan diğerine çevresindeki engellere çarpmadan pürüzsüz ve güvenli bir şekilde hareket etmesini sağlar.

Gazebo

- **Tanım:** Robotları, sensörleri ve çevre koşullarını (yerçekimi, sürtünme, rüzgar vb.) gerçeğe çok yakın bir fizik motoru ile simüle eden 3 boyutlu test ortamıdır.
- **Nasıl Yardımcı Olur?:** Yazılan kodların ve algoritmaların fiziksel robota yüklenmeden önce sanal ortamda güvenle test edilmesini sağlar. Bu sayede donanım hasarları önlenir ve prototipleme süreci hızlanır.

RViz (ROS Visualization)

- **Tanım:** Robotun anlık durumunu ve sensörlerinden gelen verileri (örneğin harita, LIDAR nokta bulutları, kamera görüntüleri) görselleştiren 3 boyutlu bir analiz aracıdır.
- **Nasıl Yardımcı Olur?:** Robotun etrafını o an nasıl "gördüğünü" ve planladığı yolları üç boyutlu olarak bizim de görmemizi sağlayarak, sensörlerin veya algoritmaların doğru çalışıp çalışmadığını anlamamıza imkan tanır.

rqt_plot

- **Tanım:** ROS konuları (topics) üzerinden akan sayısal sensör veya komut verilerini anlık olarak 2 boyutlu zaman eksenli grafiklere dönüştüren bir araçtır.
- **Nasıl Yardımcı Olur?:** Robotun hız değişimi, motor sıcaklığı veya hata payları gibi sayısal verilerinin zaman içindeki davranışını grafiksel olarak izleyerek performans analizi yapmamızı sağlar.

rqt_graph

- **Tanım:** Sistemde çalışan tüm düğümleri ve bu düğümlerin birbirleriyle hangi konular (topics) üzerinden iletişim kurduğunu gösteren dinamik bir ağ diyagramı (harita) aracıdır.
- **Nasıl Yardımcı Olur?:** İletişim kopukluklarını veya düğümlerin veriyi yanlış bir kanala gönderdiğini görsel olarak çok hızlı bir şekilde tespit etmemizi sağlayarak hata ayıklamayı (debugging) kolaylaştırır.

rosvbag

- **Tanım:** Robot aktifken konular üzerinden akan tüm sensör ve komut verilerini zaman damgalarıyla birlikte kaydeden ve sonradan oynatılmasını sağlayan kayıt mekanizmasıdır.
- **Nasıl Yardımcı Olur?:** Sahada alınan verilerin kaydedilmesini sağlar. Böylece ofise döndüğünde robot olmadan bile kaydedilen veriler tekrar oynatılarak algoritmaları geliştirme ve test etme imkanı sunar.



Çarpışma Önleyici Algoritmalar Araştırması

Çarpışma önleme algoritmaları, otonom mobil robotların planlanan hedefe ilerlerken çevrelerindeki statik (duvar, ağaç) veya dinamik (insan, diğer araçlar) engellere çarpmadan güvenli bir yörünge (trajectory) oluşturmalarını sağlayan reaktif karar mekanizmalarıdır.

Sıkça Kullanılan Algoritmalar ve Çalışma Mantıkları

1. Dinamik Pencere Yaklaşımı (Dynamic Window Approach - DWA)

Robotun anlık hızını, ivmelenme limitlerini ve fiziksel sınırlarını hesaba katar. Sadece robotun bir sonraki adımda ulaşabileceği "dinamik pencere" içindeki hız (doğrusal ve açısal) çiftlerini simüle eder. Bu simülasyonlar sonucunda hedefe en yakın, engelden en uzak ve en optimum yörüngeyi üreten hız çifti seçilerek motorlara iletilir (Fox, Burgard, & Thrun, 1997).

2. Yapay Potansiyel Alanlar (Artificial Potential Fields - APF)

Fizikteki elektromanyetik alan teorisine benzer çalışır. Robot için hedef nokta bir "çekim kuvveti" (attractive force), çevredeki engeller ise bir "itme kuvveti" (repulsive force) oluşturur. Robot üzerindeki toplam kuvvet vektörü toplanarak hareketin yönü ve hızı belirlenir. Basit ve hızlı çalışmasına rağmen, robotun yerel minimum (local minima) noktalarında sıkışıp kalma dezavantajı vardır.

3. Vektör Alan Histogramı (Vector Field Histogram - VFH)

Robot çevresini bir ızgara (grid) haritasına çevirir ve engellerin yoğunluğunu açısal bir histogram grafiği olarak çıkarır. Yüksek engellerin olduğu açılar tehlikeli, düşük olanlar ise açık kapılar olarak değerlendirilir. Robot hedefine en uygun "açık kapıdan" geçecek yönelimi seçer.

Çarpışma Önleme Bir Robot İçin Neden Önemlidir?

Çarpışma önleme, sistemin otonom olarak adlandırılabilmesi için temel bir güvenlik şartıdır. Robotun fiziksel bütünlüğünü korumak (pahalı donanım hasarlarını engellemek), çevredeki insanların güvenliğini sağlamak ve verilen görevin başarısızlıkla sonuçlanmasını önlemek için zorunludur. Çarpışan bir robot, yalnızca kendi görevini iptal etmekle kalmaz, aynı alanda çalışan diğer sistemlerin işleyişini de bozabilir.

Afet Sonrası Ortamlarda Neden Önemlidir?

Deprem, yangın veya göçük gibi afet sonrası ortamlar tamamen yapısal olmayan (unstructured) ve öngörülemez alanlardır. Zemin dengesizdir ve sürekli döküntüler oluşabilir. Böyle bir ortamda arama-kurtarma yapan bir robotun engele çarpması, mevcut döküntünün dengesini bozarak ikinci bir göçüğe sebep olabilir veya enkaz altında kalan canlıların yaşam alanını tehlikeye atabilir. Bu nedenle afet ortamlarında yüksek frekanslı çalışan, 3 boyutlu algı kullanan çok daha hassas ve reaktif algoritmalar şarttır.

Zorluk Kıyaslaması: Statik vs. Dinamik Ortam

Sadece Bir Robot ve Etrafındaki Statik Ortam

Statik ortamlarda çarpışma önleme nispeten daha kolaydır. Duvarlar, masalar veya sütunlar hareket etmediği için robot bir kez harita çıkardığında veya anlık sensör verisiyle engeli gördüğünde A* veya Dijkstra gibi global planlayıcılarla engelin etrafından dolanacak bir yol çizebilir. Gelecekte engelin nerede olacağını tahmin etmeye gerek yoktur.

Hareket Eden Objelerin/Robotların Olduđu Ortam

Dinamik ortamlarda arpışma önleme ok daha zordur. Algoritma sadece engelin řu an nerede olduđunu deđil, gelecekte nerede olacađını kestirmek zorundadır. Kalabalık ortamlarda her taraf hareketli objelerle dolduđunda, robot güvenli bir yörünge bulamaz ve olduđu yerde kilitlenip kalabilir (Freezing Robot Problem). Ayrıca hareketli bir nesneyle karşılařıldığında hesaplamaların milisaniyeler içinde yapılıp motorlara iletilmesi gerekir.

Kaynaka

ROS Documentation. (n.d.). *ROS 2 Documentation*. <https://docs.ros.org/en/humble/index.html>

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1), 23-33.

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074.

